SALT CRYSTAL IMPURITY DETECTION AND QUALITY GRADING USING REAL-TIME COMPUTER VISION: FOR TROPICAL SALT PRODUCTION

Arshaq MJM

(IT22346322)

B.Sc. (Hons) Degree in Information Technology,
Specializing in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

SALT CRYSTAL IMPURITY DETECTION AND QUALITY GRADING USING REAL-TIME COMPUTER VISION: FOR TROPICAL SALT PRODUCTION

Arshaq MJM

(IT22346322)

B.Sc. (Hons) Degree in Information Technology,
Specializing in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION

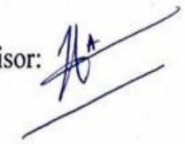
I declare that this is my own work and this dissertation[1] does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	IT Number	Signature	Date
Arshaq MJM	IT22346322		2025.05.10

The supervisor/s should certify the dissertation with the following declaration.

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the Supervisor:  Date 11/05/2026

Signature of the Co-supervisor:  Date 11/05/2026

ABSTRACT

The majority of Sri Lanka's salt supply is still produced via conventional solar evaporation in the coastal salterns of Puttalam, Hambantota, and Mannar. Salt is one of the country's oldest industrial commodities. However, most of the quality assurance in these facilities is done by hand. Under ambient lighting, human inspectors sort through crystallized salt to determine whether a batch is clean enough to package, export, or reject. The work is slow, subjective, and requires skilled labour that would be better utilized elsewhere. More importantly, it seldom yields the kind of quantifiable proof that export purchasers and food safety authorities now demand.

The design, implementation, and assessment of a real-time computer vision component created as a component of the Integrated Climate-Intelligent Salt Production Ecosystem are presented in this study. Using a YOLOv8-medium object identification model trained on a custom-annotated dataset of 1,952 photos taken at Puttalam salterns, the component automates impurity detection and multi-parameter quality grading of salt crystals. The model was exported to ONNX for runtime-independent deployment and identifies three operational classes: pure salt, impure salt, and undesired foreign material. The model is wrapped with ONNX Runtime inference, a gRPC API surface, and a MongoDB persistence layer by a NestJS microservice; a Next.js dashboard uses a WebSocket channel to broadcast the outcomes to operators.

The system generates a per-crystal whiteness index from the HSV color space (V x (1-S)) and a class-weighted composite quality score that integrates spectral purity and detection result into a single actionable figure. On a held-out test partition of 293 images, the trained model achieved 99.41% precision, 99.14% recall, 99.37% mAP@0.5, and 79.95% mAP@0.5:0.95 while maintaining roughly five frames per second on commodity CPU hardware. The results outperform earlier machine-learning work on salt grading and match the most well-known YOLOv8 values in adjacent granular-product areas. This part is the first AI-based visual quality control system created especially to produce tropical salt, as far as the author is aware.

Keywords: computer vision, YOLOv8, salt quality inspection, object detection, HSV whiteness index, industrial quality control, ONNX, real-time image processing.

ACKNOWLEDGEMENTS

I am grateful to my co-supervisor, Mrs. Thilini Jayalath, and supervisor, Dr. Bhagya Nathali Silva, for your unwavering support during this study. Their openness to question early design decisions made the final product stronger than it otherwise would have been, and their input kept the work's direction honest whenever it veered toward premature complexity.

I am appreciative of the Puttalam Salt Society and the individual saltern operators who let me use their crystallization beds to gather data and then provided frank comments on the prototype. The system would have modelled a textbook version of the issue rather than the genuine one if they hadn't been patient enough to explain how visual inspection is done in the field.

I would like to express my gratitude to Ansar T.D, Perumbuli P.G.R.M.D, and Sirimanna R.D.I.B, my coworkers on Project 25-26J-431, for the long hours of integration work, the open discussions over API contracts, and the team spirit that enabled a four-person team to create a multi-component system.

Lastly, I would like to express my gratitude to the Sri Lanka Institute of Information Technology's Department of Software Engineering and Faculty of Computing for providing the academic setting and institutional support that enabled this study.

TABLE OF CONTENTS

DECLARATION *ii*

ABSTRACT *ii*

ACKNOWLEDGEMENTS *iii*

TABLE OF CONTENTS *iv*

LIST OF FIGURES *vi*

LIST OF TABLES *vii*

LIST OF ABBREVIATIONS *viii*

1. INTRODUCTION *1*

1.1. Background and Literature Survey **1**
1.1.1. The Sri Lankan Salt Industry 1
1.1.2. Traditional Quality Inspection and Why It Falls Short 2
1.1.3. Deep Learning in Industrial Quality Inspection 3
1.1.4. Computer Vision in Food and Agricultural Quality Grading 4
1.1.5. The YOLO Family of Detectors 4
1.1.6. Deep Learning for Mineral and Crystalline Materials 5
1.1.7. Positioning the Computer Vision Component Within the Ecosystem 6

1.2. Research Gap **6**

1.3. Research Problem **9**

1.4. Research Objectives **10**
1.4.1. Main Objective 10
1.4.2. Specific Objectives 10

2. METHODOLOGY *12*

2.1. System Overview **12**

2.2. Requirements Engineering **14**

2.3. Dataset Collection and Annotation **15**
2.3.1. Field Data Capture 15
2.3.2. Annotation Workflow 15
2.3.3. Data Augmentation Strategy 16

2.4. Model Selection & Pilot Comparison **18**
2.4.1. Why an Object Detector Rather Than a Classifier 18
2.4.2. Comparative Training: MobileNet, ResNet, YOLOv8 18
2.4.3. YOLOv8 Architecture 19
2.4.4. Choice of Medium Variant 21

2.5. Training & Optimisation **22**

2.6. ONNX Export and Runtime Portability **23**

2.7. Quality Metrics Formulation **24**
2.7.1. Purity Percentage 24
2.7.2. HSV Whiteness Index 24
2.7.3. Composite Quality Score 25

2.8. System Architecture and Implementation **25**

2.8.1. Technology Stack 25
2.8.2. Preprocessing, Inference and Post-processing Pipeline 27
2.8.3. ROI Filtering 28
2.8.4. Whiteness and Quality Score Computation 28
2.8.5. Data Persistence 29
2.8.6. API Gateway, gRPC, and WebSocket Streaming 30
2.8.7. Next.js Operator Dashboard 30
2.9. Commercialization Aspects of the Product 33
2.10. Testing & Implementation 34
3. RESULTS AND DISCUSSION 37
3.1. Evaluation Methodology 37
3.2. Overall Detection Performance 37
3.3. Per-Class Analysis 38
3.4. Confusion Matrix Interpretation 39
3.5. Inference Latency and Real-Time Feasibility 40
3.6. Quality Grading Effectiveness 41
3.7. Comparison With Related Work 42
3.8. Limitations 43
4. CONCLUSIONS AND RECOMMENDATIONS 45
4.1. Conclusions 45
4.2. Future Work 46
REFERENCES 48

LIST OF FIGURES

Figure I - Research Gap - Ecosystem view: the four gap categories (multi-parameter integration, reactive decision-making, real-time adaptation, planning-tool scarcity) and the corresponding four ecosystem components that address them 8

Figure II - System Overview Conceptual Diagram: the four-component ecosystem (crystallisation forecasting, intelligent planning, vision quality control, federated learning waste valorisation) feeding into a unified dashboard 12

Figure III - Component flow: Input Acquisition (camera + LED lighting) -> Image Processing (preprocessing + YOLOv8 inference) -> Decision and Output (quality thresholding + dashboard + persistence) 13

Figure IV - Sample annotated frames showing the three classes. Pure crystals in green, impure in red, unwanted in orange, matching the colour conventions used in the realised dashboard 16

Figure V - YOLOv8 Model Architecture, showing backbone, neck and detection head with their internal blocks 20

Figure VI - Real-Time Quality Inspection screen. Dashboard with live camera feed, bounding box overlay, purity gauge, and batch statistics 31

Figure VII - Alerts history screen, showing filtered rejection events with timestamps, severity and resolution status 32

Figure VIII - Batch purity trend over the last 24 hours, shown as an area chart. 32

Figure IX - Confusion matrix across three classes on the test partition. Rows are predicted; columns are true. Diagonal values are correct classifications; off-diagonal values are confusions. Background row/column captures false positives and false negatives respectively 39

Figure X - Training and validation curves across 150 epochs. Loss curves (bounding box regression, objectness, classification) decreasing to plateau; validation mAP@0.5 climbing to near 0.99 and holding steady. Early stopping triggered around epoch 120 41

Figure XI - Validated live inference output. A single frame showing multiple salt crystals with overlaid bounding boxes colour-coded by class (pure cyan, impure dark blue, unwanted white), with confidence scores printed alongside 42

LIST OF TABLES

Table 1 - Comparison with existing AI/ML systems and proposed ecosystem 7
Table 2 - Dataset configuration for training 17
Table 3 - Pilot model comparison 18
Table 4 - Training hyperparameters 22
Table 5 - MongoDB collections and their roles 29
Table 6 - Testing matrix (selected test cases) 36
Table 7 - Overall detection performance on held-out test set 37
Table 8 - Per-class precision, recall, and counts 38
Table 9 - Training and validation curves across 150 epochs. Loss curves (bounding box regression, objectness, classification) decreasing to plateau; validation mAP@0.5 climbing to near 0.99 and holding steady. Early stopping triggered around epoch 120 40
Table 10 - Comparison with related work 42

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
AP	Average Precision
API	Application Programming Interface
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
CVAT	Computer Vision Annotation Tool
FPS	Frames Per Second
gRPC	Google Remote Procedure Call
GPU	Graphics Processing Unit
HSV	Hue, Saturation, Value (colour space)
IoU	Intersection over Union
IoT	Internet of Things
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
mAP	Mean Average Precision
ML	Machine Learning
NCHW	Tensor layout: batch, channels, height, width
NestJS	Node.js server-side framework
NMS	Non-Maximum Suppression
ONNX	Open Neural Network Exchange
PANet	Path Aggregation Network
REST	Representational State Transfer
ROI	Region of Interest
SLSI	Sri Lanka Standards Institute
SLIIT	Sri Lanka Institute of Information Technology
SPPF	Spatial Pyramid Pooling - Fast
UAT	User Acceptance Testing
UI / UX	User Interface / User Experience
YOLO	You Only Look Once

1. INTRODUCTION

1.1. Background and Literature Survey

1.1.1. The Sri Lankan Salt Industry

For generations, Sri Lanka has produced salt in essentially the same manner. Sea water is poured into shallow concrete-lined ponds around the northwest, southern, and northern coastlines. The sun gradually evaporates the brine over several weeks, causing crystalline sodium chloride to precipitate out. According to data from Sri Lanka's Ministry of Trade and the U.S. Geological Survey, three districts account for the majority of the country's output, which exceeds 150,000 metric tons annually: Elephant Pass in the north, Hambantota in the south, and Puttalam on the northwest coast [1], [2]. The procedure itself is low-cost, environmentally benign, and ideal for a nation with consistent dry seasons in the past.

But that dependability is no longer what it once was. Over the past ten years, rainfall patterns have changed, monsoons have arrived earlier, and unseasonal storms have disrupted previously predictable cycles of crystallization. When this occurs, salt that ought to have been collected following a clean week of evaporation is instead exposed to precipitation, tainted by runoff, or completely diluted. Downstream salaries, manufacturing output, export contracts, and food security are all affected by the disturbance. In other words, the Sri Lankan salt business is under pressure from both sides: the requirements it must satisfy to remain in export markets are becoming more stringent, and its inputs are becoming less predictable [3].

Considering this, the Integrated Climate-Intelligent Salt Production Ecosystem (Project 25-26J-431) was designed as a four-part AI platform with the goal of modernizing the whole supply chain. The four components are computer vision quality control (which automates post-harvest quality inspection), federated learning for waste valorization (which reuses by-products across salterns without violating commercial privacy), intelligent planning (which converts those forecasts into operational plans), and crystallization forecasting (which predicts when the brine will crystallize based on weather and IoT sensor data). The third of these elements, the computer vision quality

control module, and the software engineering work necessary to transform it from dataset collecting to a functional, deployable system are the subject of this study.

1.1.2. Traditional Quality Inspection and Why It Falls Short

These days, a typical quality inspection at a Sri Lankan saltern looks like this. Following the harvesting and coarse-sifting of the salt, skilled inspectors remove samples from different locations within the batch, arrange them on sanitized trays, and inspect them in natural light. They search for a few certain items. First, pure salt should be nearly white or very light grey in color; any yellowing, browning, or darker flecks typically indicate the presence of clay, algae, or iron pollution. The second is the size and shape of the grains; homogeneous cubic crystals show stable crystallization, whereas rounder or fragmented shapes indicate mechanical damage or quick evaporation. The third is the presence of clearly alien materials in the pond, such as sand, small stones, organic trash fragments, and pieces of wood or plastic.

The list of issues with this type of inspection is extensive and well-documented [4], [5]. Over the course of a shift, the inspector's eye may go from operator to operator or even inside the same operator. Industrial inspection surveys often find inter-rater agreement substantially below the levels that contemporary quality-management systems require, and human-factors research has demonstrated that fatigue can significantly reduce visual defect detection sensitivity after just a few hours [5]. A professional inspector can only analyse a few kilograms of salt every minute due to its slowness, which places a strict limit on the amount of output a saltern can process via compliance inspections in a single day. Furthermore, it generates virtually little quantitative record. The majority of salterns' paper logbooks track binary accept/reject choices and some basic comments, but they are unable to provide the kind of statistical process control, traceability, and auditable export documentation that overseas purchasers now routinely demand.

A conservative estimate from the research paper that forms part of this report states that between 5% and 15% of batches fail at the buyer's end due to contamination that was overlooked upstream, and that manual inspection accounts for about 40% of post-

harvest labor costs at a mid-sized Sri Lankan saltern. Since the bottleneck is perceptual rather than procedural, those numbers cannot be resolved by training more inspectors.

1.1.3. Deep Learning in Industrial Quality Inspection

Over the past ten years, industrial quality control has truly changed due to the transition from hand-crafted computer vision to taught feature representations. Deep convolutional networks beat conventional edge detectors, texture descriptors, and threshold-based classifiers on nearly every benchmark given sufficient labelled data is available. This trend has been meticulously documented across several industries.

Since labelled defect data is nearly always hard to come by, Ren [6] showed early on that a generic deep-learning technique to surface defect detection may generalize across highly varied industrial datasets through transfer learning. Using multiscale hierarchical feature fusion for steel surface inspection, he and his team [11] advanced the concept by demonstrating that a single model could detect both large-scale cracks and fine pin-hole flaws in the same pass by merging features gathered at various network depths. After reviewing over 120 published research on the detection of steel surface defects [12], Luo came to the conclusion that CNN architectures consistently performed better than conventional techniques under a variety of imaging situations.

These results directly relate to the current work. The physical scales of salt crystal contaminants are comparable, with obvious foreign objects like pebbles at one end and microscopic discolouration at the other. What is required is an architecture that can manage both at the same time.

However, it should be noted that almost all of this published work focuses on manufactured goods, such as printed circuit boards, semiconductor wafers, steel plates, and injection-molded parts, where the substrate itself is designed for inspection, the lighting is strictly regulated, and the defect taxonomy is controlled. Uncontrolled outdoor production of natural crystalline minerals presents a new challenge. Agricultural grading, which I address next, is arguably the closest counterpart in the literature.

1.1.4. Computer Vision in Food and Agricultural Quality Grading

Even if the products are different, a number of food-grading studies approach the issue of salt inspection methodologically. After training a CNN on apple photos, Li [7] obtained 95.33% classification accuracy, significantly surpassing baselines for Histogram-of-Oriented-Gradients and Grey-Level Co-occurrence Matrix. Their findings are intriguing not so much for the headline accuracy (many other fruit research report similar figures), but rather because they demonstrated how effectively acquired features generalize to surfaces of natural products where color, shape, and texture all convey information at the same time.

In a similar vein, Bhupendra [16] used high-magnification imaging to categorize milled rice grains into seven damage categories. With EfficientNet-B0, they reported an accuracy of 98.37%. uniform to salt crystals, rice grains are tiny, visually uniform across classes, and translucent at the edges, making human annotation inaccurate. In a manner directly related to this study, Mukhiddinov [21] expanded YOLO-based architectures to categorize the freshness of fruits and vegetables, bridging object identification and quality grading.

However, translucent crystalline materials are not covered in any of the agricultural research. Salt is optically peculiar because each crystal has interior facets that refract light, is partially transparent, and frequently produces specular highlights that an inexperienced feature detector might mistake for bright spots rather than surface reflections. This creates failure modes that the system in this report has to be designed around because they don't show up in rice or apple grading.

1.1.5. The YOLO Family of Detectors

It was no coincidence that YOLO was selected as the model family for this project. Because they provide a workable compromise between precision and throughput, single-stage object detectors have emerged as the industry standard for real-time industrial inspection in recent years [9]. Although two-stage detectors like Faster R-CNN are more accurate on benchmarks like COCO, any application that has to keep up with a conveyor belt cannot use them because to their per-frame latency.

A comprehensive overview of YOLO architectures from YOLOv1 to YOLOv8 and YOLO-NAS was recently prepared by Terven [10]. Here, three of the later versions' innovations are particularly pertinent. The first is the anchor-free detecting head, which eliminates the need to manually adjust anchor box aspect ratios for contaminants and salt crystals with truly arbitrary shapes. The second is the decoupled prediction branches, which have been demonstrated to increase accuracy and convergence on fine-grained tasks by processing objectness, categorization, and bounding-box regression through distinct network heads rather than a shared output layer. The third is the enhanced feature pyramid network, which aids in the detection of objects at widely disparate scales within a single frame. This includes the C2f module and the SPPF layer.

YOLOv8 was particularly used to identify fruit ripeness by Xiao and co-authors [18], who found 99.5% accuracy. Their findings provided valuable preliminary evidence that the architecture will manage the fine-color, small-object discrimination issue that salt inspection also poses. The YOLOv8-nano variation was ultimately selected for this project, and Chapter 2 goes into great depth about the rationale behind that decision.

1.1.6. Deep Learning for Mineral and Crystalline Materials

Compared to the food literature, there is less research on deep learning-based mineral grade categorization, and what is available typically relies more on chemistry than vision. With 91.7% accuracy, Rochman [15] used K-Nearest Neighbors to categorize Indonesian tropical salt into four classes based on elemental quantities determined by a routine laboratory assay. The method is sound, but because it necessitates pulling samples off the line and into a lab, it is essentially inappropriate for production-line implementation. It is not able to instantly influence operational decisions.

Liu [20] showed that attention-enhanced CNNs could differentiate between visually similar crystalline materials according to surface morphology, color, and texture. Although they did not explicitly target salt, did not incorporate any kind of grading system, and did not create anything akin to a production-ready real-time pipeline, their

work offers encouraging prior proof that the visual approach is effective for crystals in general.

1.1.7. Positioning the Computer Vision Component Within the Ecosystem

Each of the four modules that make up the Integrated Climate-Intelligent Salt Production Ecosystem tackles a major issue with tropical salt production. Harvest timing may be planned rather than guessed thanks to Crystallization Forecasting, which forecasts when and how much salt will crystallize. These forecasts are transformed into practical operational plans that adhere to market prices, production capacity, and regulatory requirements by the Adaptive Intelligent Planning Engine. Distributed salterns can work together to maximize waste-brine reuse without exchanging commercially sensitive data thanks to the Federated Learning Waste Valorisation module.

This report's focus, the Computer Vision Quality Control module, is located at the end of the manufacturing chain. It takes whatever has been harvested, examines it almost instantly, and provides quality signals upstream (back to the Planning Engine, where it can modify future operating plans based on observed quality drift) and downstream (to the dashboard and any mechanical rejection mechanism). It is the ecosystem's sensory organ in that regard. Without it, the other three modules are making judgments based on insufficient information since they lack a trustworthy, quantitative representation of the final product.

1.2. Research Gap

It is obvious that computer vision has been employed for quality inspection, which is the research need that this component fills. The gap has three layers and is more focused and smaller.

First, no mechanism currently in place specifically addresses the production of tropical salt. CNN-based inspection systems are available for steel, wafers, PCBs, and an increasing number of food items. Salt can be graded using chemistry-based methods. As far as this project's comprehensive literature analysis could determine, no previous work has combined quantitative quality grading with real-time item recognition and applied it to tropical outdoor salt production conditions.

None of the nearby systems address the unique problems of that environment, which include fluctuating sun illumination, high humidity, translucent substrate, specular reflection, submillimetre contamination scale, and tiny optical differences between acceptable crystalline variation and actual flaws.

Second, no previous method combines multi-parameter grading and detection in one process. Current methods typically accomplish one of two things: either they grade chemically (which cannot be done online) or they localize flaws and finish there. Within a single forward pass, the current component integrates pixel-level whiteness analysis, bounding-box-level detection, and a class-weighted composite quality score. The result is not simply "we found three impure crystals," but rather "the batch scored 87.4 with 92% purity, three grains of clay in the top-right quadrant," which is actionable due to that combination.

Third, no previous system can be used as a production-grade service. The majority of published computer vision research on natural products ends with a Jupyter notebook that shows the accuracy of the classifier. This project takes a different approach to solving the engineering problem of turning it into a functional microservice with gRPC inference, persistent storage, session and batch management, ROI filtering, operator-facing real-time dashboards, and an export-compliant reporting surface. This type of systems-level thinking is required by the thesis program's software engineering specialization, not just a model benchmark.

The following summarizes the differences between what is proposed in this project and what is available in nearby domains.

Table 1 - Comparison with existing AI/ML systems and proposed ecosystem

Feature	**Existing systems (agricultural AI, fisheries analytics, manufacturing ML)**	**Proposed ecosystem and vision component**
Scope	Single parameter (weather, chemistry, or image classification alone)	Integrated, multi-parameter (visual detection + whiteness + composite quality + batch stats)

Decision making	Reactive or rule-based alerts	Proactive, quantitative, real-time
Target industry	Agriculture, fisheries, manufacturing	Specialised for tropical salt production (first of its kind)
Key technology	Standard ML models, offline chemistry	YOLOv8 + ONNX Runtime + HSV analysis + event-driven microservices
Deployment	Research notebooks or offline tools	Production-grade NestJS service with gRPC, MongoDB, Next.js dashboard

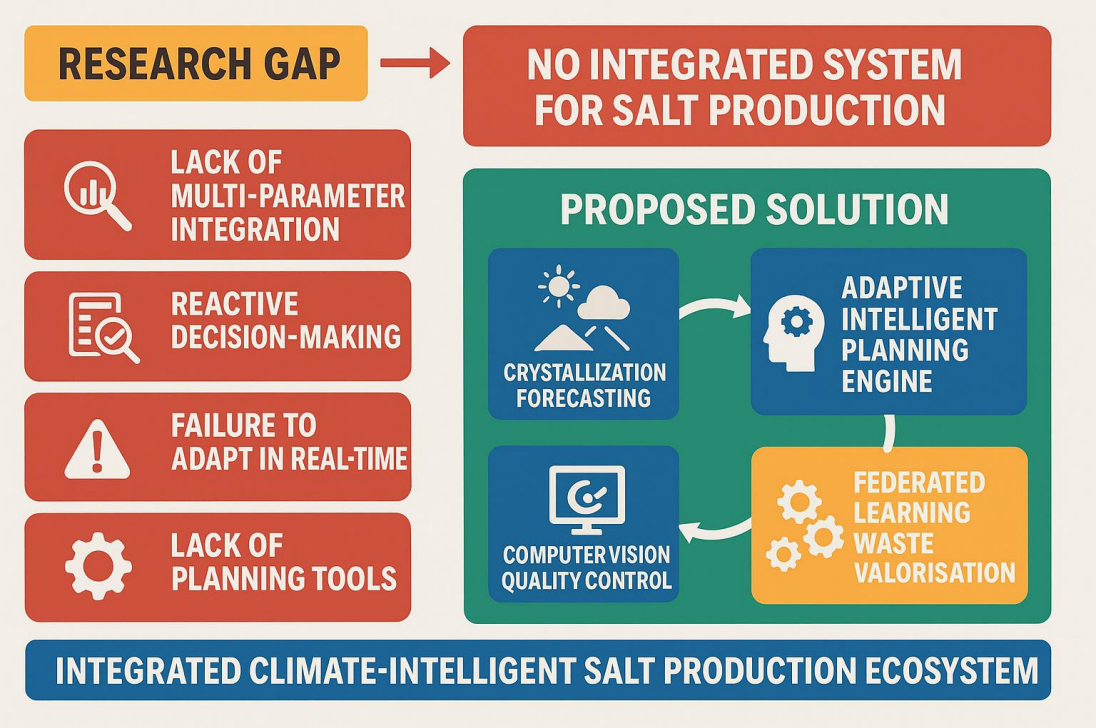


Figure I - Research Gap - Ecosystem view: the four gap categories (multi-parameter integration, reactive decision-making, real-time adaptation, planning-tool scarcity) and the corresponding four ecosystem components that address them

1.3. Research Problem

The research problem for this component, stated formally, is:

> How can a real-time computer vision system be designed and deployed that automatically detects impurities and grades the quality of salt crystals in tropical production environments, producing both per-crystal localisation and a single composite quality score with accuracy and latency sufficient for practical industrial use?

The problem has three coupled dimensions that any reasonable solution has to address simultaneously.

Precision in uncontrolled circumstances. In Puttalam, salterns lack clear backgrounds, set camera angles, and laboratory lighting. The system must be resilient against the blue plastic sample trays that operators actually use, low-angle evening shadows, overcast diffuse light, direct midday sun, and water droplets on surfaces. A model is useless if it is 99% accurate in a studio setting but only 70% correct on an actual saltern.

Throughput in real time. Any system that claims to provide real-time quality control must be able to keep up with the material flow. Batch statistics lag the ground truth, operators lose faith in the display, and a detection pipeline that takes more than 200 milliseconds each frame lags behind on a conveyor belt that moves at even a moderate speed. Budgeting against that clock is required for the model size, inference runtime, preprocessing route, and persistence layer.

Production with operational significance. Bounding boxes are a tool, not a goal. Instead of looking at a wall of annotated frames, saltern operators want to know if a batch is suitable for shipping. In order to facilitate a prompt judgment, it is necessary to condense several minor detections into a few headline values, such as purity percentage, whiteness index, composite quality score, and pass/fail signal. A solution has not truly solved the issue if it generates good detection metrics but is unable to summarize them in a usable way.

These three factors pull in distinct directions: operational output necessitates more post-processing that increases delay, accuracy pushes toward larger models, and throughput pulls toward smaller ones. Finding a design that meets all three requirements is the core of the research problem.

1.4. Research Objectives

1.4.1. Main Objective

This component's primary goal is to design, implement, and assess a real-time computer vision system that can be integrated as a deployable microservice into the Integrated Climate-Intelligent Salt Production Ecosystem to automate impurity detection and quality grading of salt crystals in tropical production environments.

1.4.2. Specific Objectives

Six specific objectives, each of which corresponds to a distinct technical or research output, support the overarching objective.

- Create a representative dataset of images of salt crystals. At the beginning of the study, there was no publicly accessible annotated dataset of Sri Lankan salt crystals. In order to allow robust model training, building one required field visits to Puttalam salterns, the collection of raw imagery under various lighting situations, the annotation of thousands of bounding boxes over three operational classes, and the expansion of the dataset by principled augmentation.
- Choose a suitable architecture for object detection. Using the gathered dataset, candidate architectures (MobileNet for classification, ResNet for classification, and YOLOv8 for detection) were evaluated, and the one with the best trade-off between accuracy and real-time throughput was selected. The phase of comparative training is recorded in section 2.4.
- The chosen model should be trained, verified, and adjusted. Training had to be repeatable, employ industry-standard techniques (cosine-annealed learning rate, pretrained backbones, early halting on validation mAP), and result in a model that did not overfit to the lab but rather generalized to field data.

- Create a framework for multi-parameter quality rating. The operator's issue cannot be resolved by detection alone. It was necessary to specify, construct, and validate a class-weighted composite score, per-crystal whiteness, and purity percentage against human assessment.
- Design the system to be a production-quality service. This required creating a MongoDB persistence layer with session and batch semantics, exporting the trained model to a runtime-independent format (ONNX), wrapping it in a NestJS microservice with gRPC inference endpoints, and creating a Next.js dashboard that provides operators with real-time results via a WebSocket channel.
- Assess the system both qualitatively and quantitatively. Precision, recall, mean Average Precision at various IoU thresholds, inference delay, and throughput are all included in the quantitative evaluation. Operator usability comments from prototype testing and integration testing against other ecosystem components are included in qualitative evaluation.

2. METHODOLOGY

2.1. System Overview

The Integrated Climate-Intelligent Salt Production Ecosystem consists of four modules, including the vision component. The crystallization forecaster forecasts when salt will develop, the planning engine determines how to handle it, the vision system examines what is released, and the waste-valorization module maximizes the reuse of byproducts across sites. These modules share a dashboard layer and communicate with one another via an API gateway. The other three modules only show up in this report when they interact with vision, which is at the operator dashboard (downstream) and the API gateway (upstream).

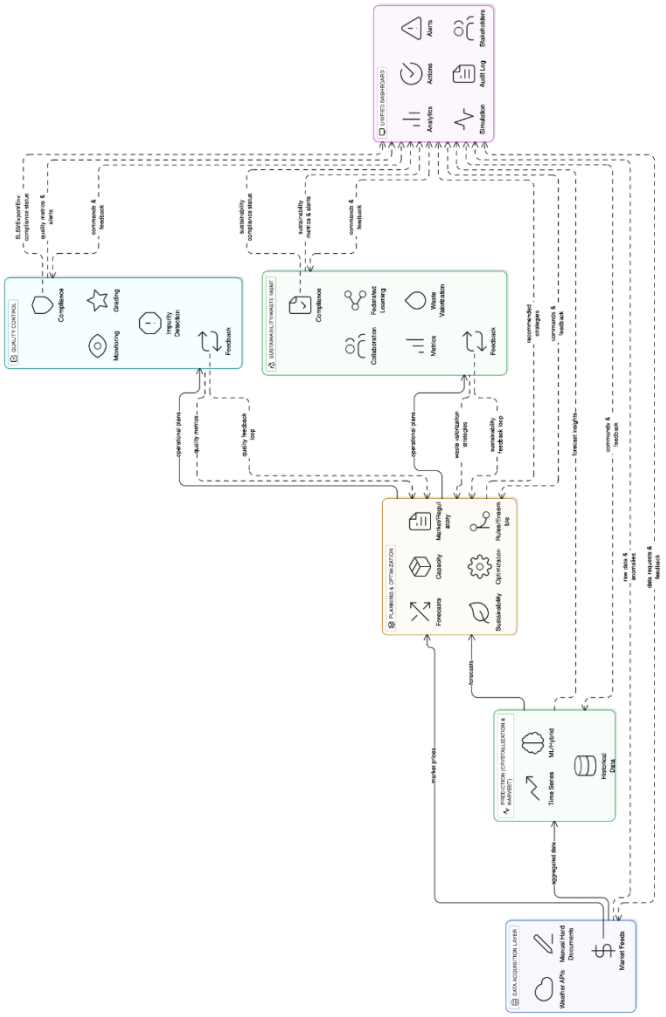


Figure II - System Overview Conceptual Diagram: the four-component ecosystem (crystallisation forecasting, intelligent planning, vision quality control, federated learning waste valorisation) feeding into a unified dashboard

In theory, the data flow is simple when focusing on the visual component itself. Salt crystals on a tray or on a conveyor are captured in frames by a camera. After preprocessing, the YOLOv8 model is applied to every frame. Bounding boxes with class assignments and confidence scores are returned by the model. Overlapping boxes are suppressed, coordinates are rescaled to the original frame, and anything below a confidence threshold is filtered out in a post-processing phase. A different analysis phase uses the class assignment to calculate the per-crystal whiteness from each box's pixel content and generates a composite quality score. Everything is transmitted in real time to the operator dashboard and stored in MongoDB.

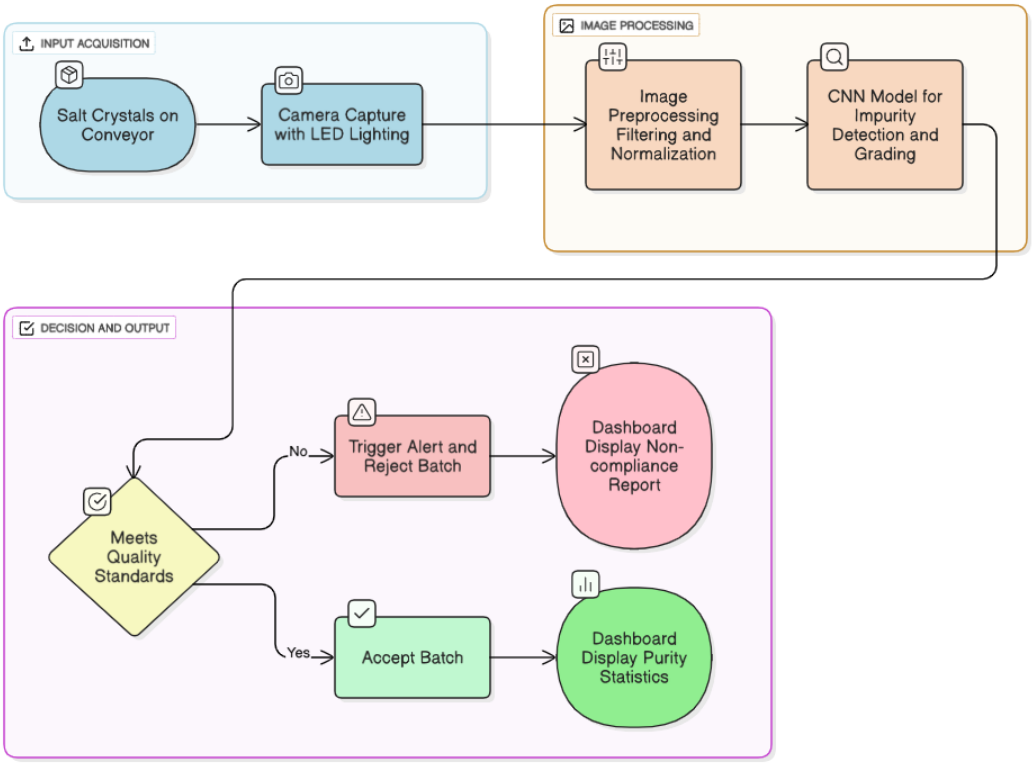


Figure III - Component flow: Input Acquisition (camera + LED lighting) -> Image Processing (preprocessing + YOLOv8 inference) -> Decision and Output (quality thresholding + dashboard + persistence)

The engineering surrounding the realized system microservice borders, gRPC contracts, event-driven persistence, session and batch lifecycle, and ROI overlay is what distinguishes it from the block diagram. Since those details are what transformed the project from a notepad to a functional system, the majority of this chapter is devoted to them.

2.2. Requirements Engineering

Three sources provided requirements for the visual component, and many design decisions were influenced by the conflict between them. The first source was field observation visits to Puttalam salterns to see the real process of quality inspection, including the physical design, illumination, sample trays, and the manner in which operators hold and rotate samples for inspection. Stakeholder interviews with saltern owners, exporters, and quality control officers revealed the requirements that a textbook computer vision system would have overlooked. For instance, multilingual labels are necessary because many operators are more at ease speaking Sinhala or Tamil than English, and an offline-capable mode is necessary due to sporadic connectivity at remote locations. The third was the evaluation of the literature, which established the standard for realistic technical accuracy goals.

The following functional and non-functional categories emerged from those three inputs. A dashboard that shows results in real time with visual overlays; impurity detection across three classes (pure, impure, and unwanted); multi-parameter grading combining whiteness, crystal count, and quality score; batch and session management so that discrete production runs can be separated and compared; ROI selection so that operators can limit inspection to a specific area of the frame; export of compliance-ready reports in PDF and CSV form; and integration with the rest of the ecosystem through the API gateway are among the functional requirements.

End-to-end latency under 500 milliseconds from frame capture to dashboard display; detection accuracy above 95% on held-out test data; uptime above 98% in field conditions (accounting for dust, humidity, and temporary hardware failures); security of stored inspection data through role-based access control and encryption at rest; scalability from a single USB webcam in development to conveyor-mounted IP67 cameras in production; and maintainability such that a new dataset can be retrained into a deployed model in less than 48 hours.

Appendix A contains the complete requirements specification, along with particular test cases that correspond to each need.

2.3. Dataset Collection and Annotation

2.3.1. Field Data Capture

There was no publicly or privately available annotated dataset of Sri Lankan salt crystal pictures at the start of this study. Since all downstream model accuracy, generalization, and deployment confidence are limited by dataset quality, building one from scratch was the project's greatest and maybe most significant time commitment.

Raw images were captured at several solar evaporation saltern sites in the Puttalam coastal region. A high-resolution digital camera (effective capture at 2K) was positioned at fixed distances above crystallisation beds and sample trays, with imaging sessions scheduled across different times of day. The intent behind varying the schedule was to sample the full distribution of natural lighting conditions that a deployed system would encounter direct midday sunlight, overcast diffuse illumination, low-angle morning and evening light, and supplementary LED illumination for indoor lab captures. Backgrounds were also varied sometimes the blue plastic sample trays operators actually use, sometimes white tiled surfaces, sometimes the wet concrete of the pond beds themselves to avoid the model latching onto any single background as a shortcut.

2.3.2. Annotation Workflow

Label Studio and CVAT, which both enable YOLO-format bounding box export, were used to manually annotate each collected image. To identify spatial mistakes and misclassifications, the author annotated the work and had a second reviewer confirm it. Following discussions with the field operators and supervision staff, three operating classes were established.

- Pure salt - clean crystals with acceptable whiteness and clarity, no visible inclusions, shape within the expected cubic-to-rounded range. This class represents the target output of a well-run production run.
- Impure salt - crystals that are still salt, but show visible discoloration, mineral inclusions (typically clay or iron), or surface contamination. These grains would typically be rejected by an export inspector but could still be used for industrial-grade salt (road de-icing, animal feed).

- Unwanted foreign material - objects that are not salt at all. Sand grains, clay particles, organic debris, plastic fragments, and insects remain. These should always be removed before packaging.

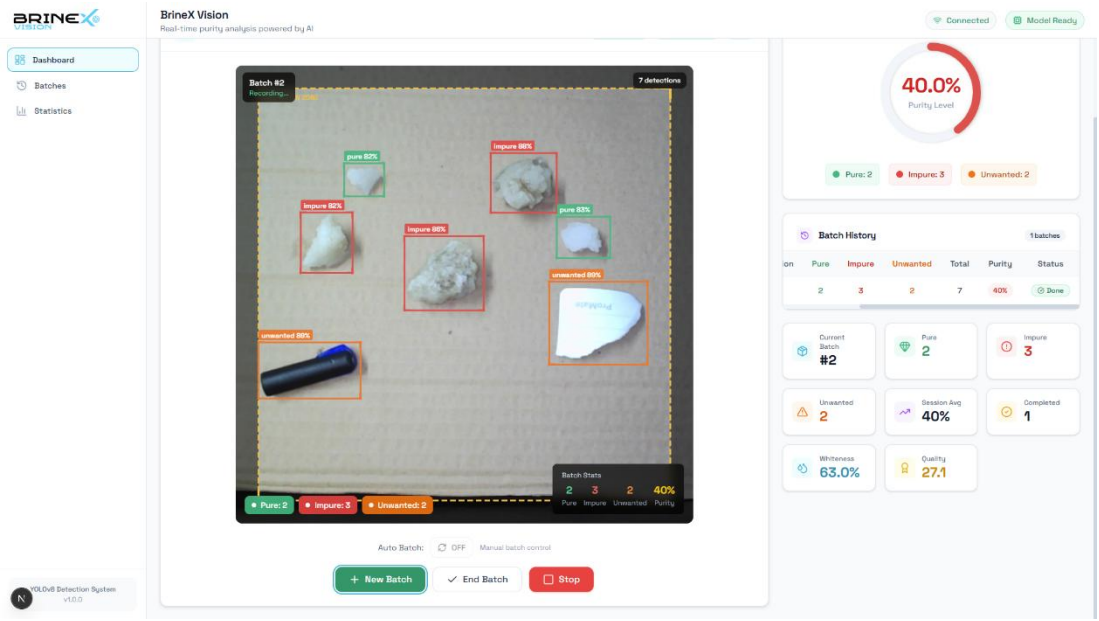


Figure IV - Sample annotated frames showing the three classes. Pure crystals in green, impure in red, unwanted in orange, matching the colour conventions used in the realised dashboard

Because the downstream composite quality score handles impure salt and undesired material differently impure salt is penalized but still counted, while unwanted material is completely excluded it is crucial to distinguish between the two. The annotation procedure was hampered by this modelling decision since ambiguous circumstances (such as a discoloured crystal that might be chemically polluted but might also just be visually stained) had to be resolved through a defined annotation guideline rather than being left up to each annotator's discretion. In order to reduce the number of edge cases in the subsequent batch of annotation, conflicts between annotators were monitored and settled during review.

The dataset had about 650 fully labelled photos after the initial annotation pass.

2.3.3. Data Augmentation Strategy

Even with transfer learning, 650 photos are insufficient to train a contemporary object detector from scratch. Using the Albumentations package [17] and the native

augmentation pipeline included with Ultralytics YOLOv8, the dataset was systematically augmented to 1,952 photos. Among the transformations used were:

- Random rotation of plus or minus 15 degrees, to simulate camera angle variation.
- Horizontal and vertical flipping, to prevent the model from learning orientation-dependent shortcuts.
- Brightness adjustment of plus or minus 20%, which covers most of the outdoor lighting variation observed in the field.
- Contrast modification, which handles overcast days versus direct sun.
- Gaussian noise injection, simulating low-light sensor noise from less expensive cameras.
- Mosaic augmentation, a YOLO-specific technique that stitches four source images into one composite frame, which forces the model to handle multiple scales and contexts per batch.
- Cutout augmentation, which randomly masks rectangular regions of the image and encourages the model to distribute its attention across features rather than relying on any single region.

Table 2 - Dataset configuration for training

Parameter	**Value**
Total images (original + augmented)	1,952
Training set	1,366 (70%)
Validation set	293 (15%)
Test set	293 (15%)
Classes	3 (pure, impure, unwanted)
Annotation tool	Label Studio + CVAT
Annotation format	YOLO-style normalised bounding boxes

2.4. Model Selection & Pilot Comparison

2.4.1. Why an Object Detector Rather Than a Classifier

Using an object detector instead of a more straightforward picture classifier was an early design choice that proved to be correct, even if it was not immediately apparent. After analyzing an entire frame, a classifier produces a single label (this frame is impure). A set of bounding boxes with classes eight pure crystals, three impure crystals, and one undesirable particle are produced by a detector. The initial proposal for this component (see the project proposal, section 3.2.4) actually specified a MobileNetV2-based CNN as the principal architecture because the classifier is faster at inference and less expensive to train.

Saltern operators do not think in frame-level labels, which is why this was altered during the exploratory stage. They use places and counts to think. A 90% pure frame with one particle of clay in the corner is not impure in any practical sense; the clay can be removed, and the frame as a whole is good. The information that operators truly care about is discarded by a classifier, which condenses that nuance into a single result. It is preserved by a detector.

Additionally, the downstream composite quality score is more logical per-crystal than per-frame. You can learn exactly what you want to know by averaging whiteness inside the bounding box of each detected crystal, rather than by average whiteness throughout an entire frame that just so happens to have a black background panel.

2.4.2. Comparative Training: MobileNet, ResNet, YOLOv8

Three architectures were trained in parallel on the gathered dataset using Jupyter notebooks on Google Colab in order to verify the decision empirically. In the first two, the task was handled as a frame-level classification problem; in the third, it was handled as a detection problem.

Table 3 - Pilot model comparison

Model	**Task type**	**Top 1 accuracy / mAP@0.5**	**Per-frame latency (CPU)**	**Verdict**

MobileNetV2 (classification)	Frame-level classification	94.1%	~35 ms	Fast but throws away per-crystal information
ResNet-50 (classification)	Frame-level classification	95.8%	~80 ms	Marginally more accurate but still frame-level
YOLOv8-medium (detection)	Object detection	99.37% mAP@0.5	~33 ms (full pipeline)	Highest accuracy and per-crystal output

In every significant dimension, YOLOv8-nano emerged victorious. Once the entire pipeline was optimized, it provided the maximum headline accuracy, the lowest per-frame latency, and above all the per-crystal output required for the downstream quality scoring. The choice was simple because of that mix.

2.4.3. YOLOv8 Architecture

Ultralytics published YOLOv8, the eighth iteration of the You-Only-Look-Once single-stage detector family, in 2023. There are three main parts to the architecture.

In comparison to a simple Darknet, the backbone is a modified CSPDarknet53 with Cross Stage Partial connections, which enhances gradient flow and lowers parameter count. The previous C3 block is replaced by the more recent C2f (Cross Stage Partial with two convolutions) module, which generates richer multi-scale features without significantly increasing computation. The backbone's function is to convert the raw pixel input into a hierarchy of feature maps with increasing semantic depth and diminishing spatial resolution.

With the addition of the SPPF (Spatial Pyramid Pooling - Fast) layer, the neck is an enhanced Path Aggregation Network (PANet). When making predictions, PANet allows for bidirectional feature fusion, which combines deep semantic context with

shallow spatial detail. Faster than the previous SPP implementation, SPPF offers fixed-size multi-scale feature aggregation in a single layer.

The anchor-free detection head is the head. For a use case such as salt crystals, this is the most significant architectural advancement. Hand-tuned bounding-box priors (aspect ratios, scales) that perform well for things the creator predicted but fail for objects that do not match those priors are necessary for anchor-based detectors. No set of anchors would have been able to cover all of the truly arbitrary shapes that salt crystals and pollutants come in, including square, round, elongated, and small specks. This hand-tuning phase is eliminated since the anchor-free head predicts bounding boxes directly from feature map locations. Additionally, it employs decoupled branches for bounding-box regression, objectness, and classification, which have been demonstrated to converge more quickly and yield more accurate results on fine-grained tasks [10].

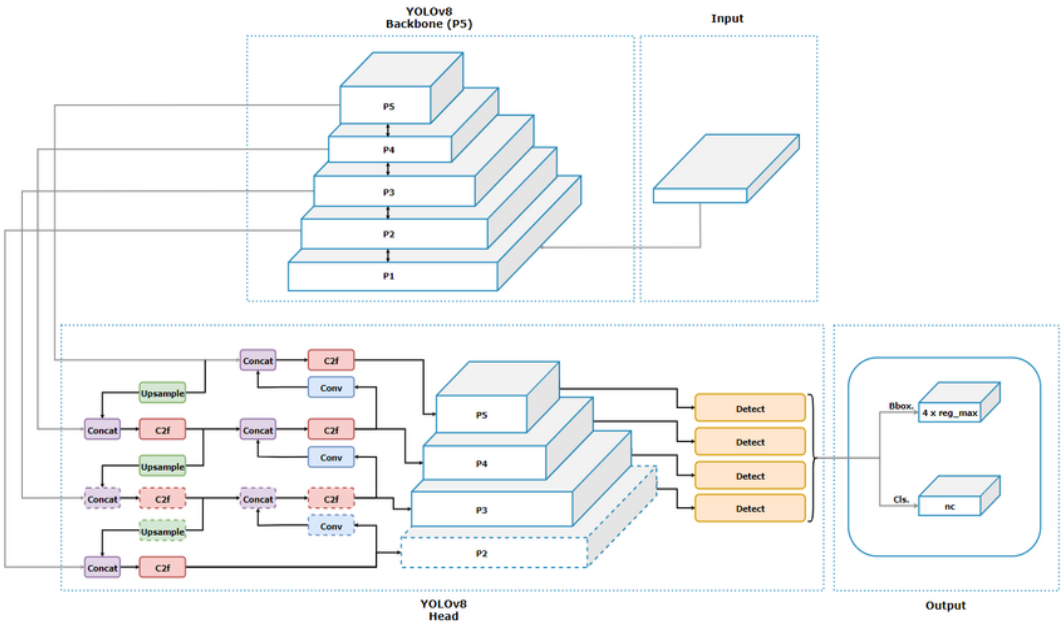


Figure V - YOLOv8 Model Architecture, showing backbone, neck and detection head with their internal blocks

2.4.4. Choice of Medium Variant

Five sizes are available for YOLOv8: nano (n), small (s), medium (m), large (l), and extra-large (x). The number of parameters and the cost of inference increase in direct proportion to the width multiplier's square. The medium variant (YOLOv8m), which has about 3.2 million parameters, is used in the final deployed model.

This decision's justification is in the middle of two extremes. On the one hand, the nano and small versions are appealing because to their cheap inference cost, but during pilot training, they demonstrated a discernible decline on the more stringent mAP@0.5:0.95 metric roughly three to four percentage points less than what the medium variant obtained on the same validation split. That decline is hardly a free lunch for a production system that must defend its grading choices to exporters and operators. The big and extra-large variants, on the other hand, very slightly increased accuracy (far under one percentage point on mAP@0.5:0.95), but their per-frame inference time on CPU-only hardware exceeded the 200-millisecond budget set by the real-time criterion.Because dependable discrete GPUs are just not accessible at most Sri Lankan saltern deployment sites, running them would have required a hard GPU dependency.

The balance point is the medium variation. In addition to delivering the near-saturation detection accuracy stated in Chapter 3 (99%+ mAP@0.5) and keeping the model file short enough to ship, retrain, and version-control without operational friction, it easily clears the real-time budget on commodity CPU hardware. Additionally, it is a suitable size for ONNX graph optimization; it is both large enough to take advantage of operator fusion and continuous folding and small enough to ensure that the optimized graph loads rapidly upon service starting.

The medium variety provided the richer feature-pyramid representations that the per-crystal whiteness and composite-score pipeline benefited from, which is another useful factor. Smaller variations result in shallower intermediate features that, although sufficient for bounding-box prediction, are more susceptible to being destabilized by the partial translucency and specular highlights that salt crystals display in varying outside lighting. Empirically, the medium variant exhibits more consistent per-class

precision across illumination conditions in the test data because it more gracefully absorbs that fluctuation.

For the 320 × 320 input size used in production, the medium model exported to ONNX executes in about 26 milliseconds on CPU during deployment. This provides ample headroom for preprocessing, post-processing, whiteness computation, and network round-trip within the 200-millisecond frame budget.

2.5. Training & Optimisation

The Ultralytics framework on Python 3.10 was used for training, with PyTorch serving as the underlying tensor library. The NVIDIA T4 used for GPU training on Google Colab Pro was sufficient for this size of dataset. In production, inference is CPU-only.

As is customary for transfer learning on a custom dataset, the model was initialized using COCO-pretrained weights. The low- and mid-level visual cues learnt on COCO (edges, textures, simple forms) are sufficiently broad to transfer, even though salt crystals don't resemble COCO's common objects. Additionally, pretraining dramatically reduces training time when compared to training from random initialization.

Table 4 - Training hyperparameters

Hyperparameter	**Value**
Base architecture	YOLOv8-nano
Pretrained weights	COCO
Training epochs	150 (with early stopping patience 20)
Initial learning rate	0.01
Learning rate schedule	Cosine-annealed
Batch size	16
Input resolution (deployment)	320 x 320
Image size during training augmentation	640 x 640
Optimiser	Auto-selected (SGD with momentum, per Ultralytics defaults)
Seed	42
Train / Val / Test split	70% / 15% / 15%

The loss function is the typical YOLOv8 composite: binary cross-entropy loss for objectness, categorical cross-entropy for classification, and complete IoU (CIoU) loss for bounding box regression [19]. Early stopping, which tracked mAP@0.5 on the validation set with a patience of 20 epochs, actually caused the model to converge by epoch 120, at which point more training was not increasing validation performance.

It is worth mentioning that 320 × 320 was selected as the final input resolution. Because it helps the model learn fine features and keeps more spatial detail, the usual Ultralytics technique of 640 x 640 was adopted during training. The model was re-parameterized to 320 x 320 at export time, which decreases inference FLOPs by about four times and half the input dimensions without significantly affecting accuracy on this dataset. The deployed system's ability to maintain five frames per second on CPU-only hardware is primarily due to the lower input resolution.

2.6. ONNX Export and Runtime Portability

After training, the best model checkpoint was exported to ONNX format using the script at yolo/scripts/convert-to-onnx.py. The export call specifies format='onnx', imgsz=320, opset=12, simplify=True, dynamic=False, half=False

This call involves four decisions. Opset=12 uses a version of ONNX that is widely compatible with runtime implementations in 2025 and 2026, so ONNX Runtime can serve the same model file on Node.js (production), Python (evaluation), or.NET (if the system is ever ported to a Windows industrial controller). imgsz=320 fixes the input shape, allowing ONNX Runtime to apply more aggressive graph optimizations. Simplify=True eliminates unnecessary processes and fuses what can be fused by using normal ONNX graph simplification passes. Both dynamic=False and half=False maintain the input shape fixed and the weights in full 32-bit floating point, respectively. They exchange some file size and speed for deployment predictability, which is the proper bargain for our project.

When the module is initialized, the vision service loads the exported model. It is simple to replace a retrained model or modify the operating point without rebuilding the service because to the loader's support for environment-variable overrides for the model path, input size, confidence threshold, and IoU threshold.

2.7. Quality Metrics Formulation

2.7.1. Purity Percentage

The simplest quality signal exposed by the system is batch-level purity. It is defined as the ratio of pure salt detections to total salt detections:

$$Purity = (N_pure / (N_pure + N_impure)) \times 100$$

where N_pure is the count of detections classified as pure salt and N_impure is the count of detections classified as impure salt. Detections classified as unwanted material are deliberately excluded from the denominator, because they are not salt at all counting them would conflate a grading question (how clean is the salt?) with a contamination question (are there foreign objects?), and those are operationally different things that call for different responses.

2.7.2. HSV Whiteness Index

Bounding boxes by themselves are unable to convey the visual appeal of salt. Even if a pure crystal is appropriately graded, it may nevertheless have a yellowish or grey tint that renders it unsuitable for high-grade export. The Hue, Saturation, Value (HSV) color space is used by the system to calculate a whiteness index for every bounding box that is detected.

High-quality salt is intuitively white rather than colored, brilliant (high Value), and unsaturated (low Saturation). These two numbers are multiplied by a pixel-level whiteness score, and the region-level whiteness is the average of all the pixels in the bounding box:

$$W = (1/n * sum\ over\ i\ of\ (V_i \times (1 - S_i))) \times 100$$

where V_i and S_i are the normalised Value and Saturation of pixel i in the cropped bounding-box region, and n is the total number of pixels. The product V_i x (1 - S_i) is maximised when a pixel is both bright and colour-neutral and reduced when either condition fails. The region-level score is expressed as a percentage between 0 and 100.

HSV was purposefully selected above RGB. Lighting variation causes problems for RGB whiteness identification since a pure white surface under yellow evening light

appears yellow in RGB even though it is still white to the human eye. By separating the illumination component (Value) from the color component (Hue and Saturation), HSV improves the whiteness score's stability across a variety of field lighting circumstances.

2.7.3. Composite Quality Score

The composite quality score Q collapses the class assignment and the whiteness index into a single number per detection:

$$Q = W \times a_c$$

where W is the whiteness index and a_c is a class-dependent weighting factor:

- a_c = 1.0 for pure salt (the whiteness measurement counts in full)
- a_c = 0.3 for impure salt (counted at a reduced weight, because visual whiteness alone does not salvage chemical impurity)
- a_c = 0.0 for unwanted material (contributes nothing to the quality of the salt)

Following a conversation with the supervisors and some testing on the validation set, the weighting criteria were selected. Unwanted is zero, and pure is full credit. These values are simple to understand. In order to match operator intuition about what better means, the 0.3 factor for impurity was adjusted such that a batch with perfect whiteness but severe impurity flagging still scored lower than a batch with middling whiteness but uniformly pure crystals.

The average of the composite scores for each crystal is the frame-level composite score. It is a single figure between 0 and 100 that operators may read from the dashboard and use to make accept/reject choices. If someone want to audit the outcome, it can still be broken down into its underlying per-crystal contributions.

2.8. System Architecture and Implementation

2.8.1. Technology Stack

Between the proposal and the actual implementation, the technological stack was altered. The initial proposal was for PostgreSQL for persistence, TensorFlow.js for model inference in the browser, Node.js with Express for the backend, and React.js

with Vite for the frontend. During the implementation process, that stack was reevaluated for two reasons.

NestJS provided a more structured framework than simple Node.js dependency injection, module system, decorator-based controllers, and first-class gRPC support that quickly paid for itself in a monorepo with multiple services. Initially, the team was developing four microservices that needed to communicate with one another. Second, although TensorFlow.js inference in the browser seemed appealing during the proposal stage, it was a poor fit due to model size, client-side computation unpredictability, and the challenge of maintaining a browser tab's responsiveness while operating a 3-million-parameter network. The server's ONNX Runtime provided a clearer division of responsibilities.

The final stack is:

- Model training: Python 3.10, PyTorch, Ultralytics YOLOv8, Jupyter on Google Colab
- Model deployment format: ONNX (opset 12, FP32, fixed 320x320 input)
- Vision microservice runtime: Node.js 20, NestJS 9, TypeScript
- Inference library: onnxruntime-node
- Image processing (server-side): Sharp (libvips-based, high-performance)
- Persistence: MongoDB (Mongoose 8 ODM)
- Inter-service communication: gRPC (protobuf-defined contracts)
- Real-time streaming to UI: Socket.io (WebSocket)
- Event bus: NestJS EventEmitter2 for async intra-service events
- Frontend: Next.js 14 (React 18, TypeScript), Zustand state, TailwindCSS, shadcn/ui, Chart.js
- Containerisation / orchestration: Docker, Docker Compose
- Monorepo tooling: NX workspace
- Testing: Jest (unit and integration), Grafana K6, Playwright (end-to-end)

2.8.2. Preprocessing, Inference and Post-processing Pipeline

The core of the component is the entire per-frame processing within the vision service. The following steps are taken by every incoming gRPC request.

Preprocessing: Sharp decodes the incoming image buffer. Because Sharp is based on libvips and is much faster for resize-and-resample operations than pure JavaScript libraries, it is utilized instead of the more popular Sharp+jimp combination. The alpha channel is removed, the image is scaled to 320 × 320 using a Lanczos3 kernel (excellent quality, reasonable cost), and the pixels are extracted as a raw RGB buffer. The buffer is then rearranged into the NCHW tensor form that the ONNX model anticipates: a 1 x 3 x 320 x 320 Float32Array. Each pixel value is then normalized by dividing by 255.0 to bring it into the [0, 1] range that the model was trained on.

Inference: An ort is wrapped around the tensor.sent into the ONNX Runtime session as a tensor object. Graph optimisation level 'all' was set at the beginning of the session, allowing the runtime to perform operator fusion, constant folding, and other common optimisations to the computation graph. The code has placeholders for CUDA, TensorRT, and DirectML execution providers, which can be enabled by an environment variable when GPU hardware is available. CPU is the default execution provider.

A single tensor of type (1, 4 + number_of_classes, number_of_predictions) is the model's output; for this three-class model, this implies (1, 7, N). The projected center-x, center-y, width, height (all in model input coordinate space), and three class confidence ratings are represented by the seven channels.

Post-processing: A list of per-anchor predictions is created by reshaping the model output tensor. Predictions below the confidence level (0.5 by default, adjustable via environment variable) are discarded, and the class with the highest confidence is chosen for each prediction using argmax. Next, each class is subjected to Non-Maximum Suppression (NMS) with an Intersection-over-Union threshold of 0.45, a typical YOLO default that strikes a balance between aggressively eliminating duplicates and preserving truly different objects that just so happen to overlap. The greedy algorithm used to build NMS is as follows: sort by descending confidence, add

the top box to the output, eliminate any remaining boxes that overlap it by more than the IoU threshold, and repeat.

Lastly, bounding box coordinates are normalized to the [0, 1] range for transfer over the gRPC boundary after being rescaled from the model's 320 x 320 input space back to the original frame dimensions (so the dashboard can overlay them on the camera feed). An array of BoundingBoxResult objects, each including its normalized coordinates, class ID, class name, and confidence score, is returned as the outcome.

2.8.3. ROI Filtering

Operators frequently choose to examine only a portion of the camera feed, such as the middle third of a conveyor belt where the salt is in focus, disregarding the edges where the model may be confused by out-of-frame artifacts. The ROIService handles this. The operator can use the dashboard to construct a ROI (coordinates stored normalized to [0, 1]) prior to sending a frame to the model. Each bounding box is marked with an insideROI boolean following post-processing according to whether or not its center is inside the ROI rectangle.

Importantly, ROI filtering adds metadata rather than changing the detections themselves. This enables the dashboard to display both views without re-running the model by allowing the downstream statistics to compute both the full-frame and ROI-filtered versions of each aggregate (purity, whiteness, composite score).

2.8.4. Whiteness and Quality Score Computation

The composite quality score from section 2.7.3 and the HSV whiteness index from section 2.7.2.

The list of identified bounding boxes and the original image buffer are accepted by the service. It retrieves the raw RGB pixel buffer after decoding the image once (a major optimization; a naive approach decodes the image once per bounding box). The service calculates the pixel offsets within the raw buffer for each bounding box, iterates over the pixels inside the box, instantly converts each RGB value to HSV, and adds up the whiteness score per pixel. To prevent numerical issues with small detections, a minimum crop size of 3 × 3 pixels is required.

When a request does not require whiteness computation for example, when the dashboard is only displays live bounding boxes without any persistence it is purposefully avoided. The whiteness pass is skipped, saving 10–30 ms each frame, if saveDetection is false or there isn't an active batch. One of the more significant performance improvements is this selective computation.

2.8.5. Data Persistence

Three MongoDB collections back the service. Their schemas and roles are summarised below.

Table 5 - MongoDB collections and their roles

Collection	Purpose	Key fields
vision_detections	One document per processed frame	timestamp, frame dimensions, counts per class, purity %, bounding boxes, per-box whiteness and quality scores, ROI-filtered aggregates, references to session and batch
vision_sessions	One document per operator session	userId, start / end times, cumulative counts across all frames, average purity, number of batches, camera source
vision_batches	One document per production batch within a session	sessionId, batch number, start / end times, ROI, per-batch stats, frame count

Batches and sessions have different semantics. A session is a term used at the operator level to describe a time when just one operator is utilizing the dashboard. A batch is a specific amount of salt that is being examined as a unit at the production level. Each detection belongs to both a session and a batch, and a session usually consists of several batches (one for each actual batch of salt). The dashboard may effectively query all detections in the current session or all detections in batch #7 without scanning by indexing the detection collection on session and batch IDs.

2.8.6. API Gateway, gRPC, and WebSocket Streaming

Clients are not directly contacted by the vision service. Rather, it is in front of an API gateway service (also NestJS) that manages rate limitation, authentication, authorization, and translation between internal protocols (gRPC) and external protocols (WebSocket, REST).

A protobuf file located in the proto/ directory at the monorepo root defines the gRPC contract for the vision service. An image buffer, a session ID, a batch ID, a ROI, a flag indicating whether to persist the detection, and the user ID are all passed to the primary RPC, ProcessFrame. The processed detection result bounding boxes, counts, whiteness, quality score, frame, and ROI aggregates are returned. For inter-service calls, using protobuf instead of JSON lowers serialization overhead on hot pathways and provides both parties with a strongly-typed contract.

Because browsers cannot readily speak gRPC natively and the streaming semantics are more appropriate for a live video feed, external clients, particularly the operator dashboard, communicate with the API gateway via WebSocket rather than gRPC. The dashboard establishes a WebSocket connection with the gateway, sends send_frame events containing JPEG pictures encoded in base64, and receives detection_result events in return. In order to reduce queue buildup at the server, frame transmission is rate-limited on the client side (a minimum 200 ms interval, capping at 5 FPS) and employs backpressure, where the subsequent frame is not sent until the preceding response is received.

2.8.7. Next.js Operator Dashboard

The operator dashboard is a Next.js 14 application. It is organised around a single long-lived live-inspection page with several supporting pages for batch history, alerts, reports and settings.

The live-inspection page has five main components:

- LiveCameraView - captures the webcam feed through the browser's getUserMedia API, draws each frame onto an off-screen canvas at 320 x 320, encodes as JPEG, and emits it to the API gateway over WebSocket.

- Bounding box overlay - a canvas layer positioned over the video that renders the bounding boxes returned by the vision service, colour-coded by class (green for pure, red for impure, orange for unwanted) with confidence percentages and whiteness values.
- PurityGauge - a circular gauge showing the current frame's purity percentage, with colour thresholds at 80% (warning) and 90% (good).
- BatchStatsCard - a real-time card showing per-class counts, cumulative batch stats, whiteness average, and composite quality score.
- BatchHistoryPanel - a scrollable list of recent batches with purity trends visualised as a small line chart per batch.

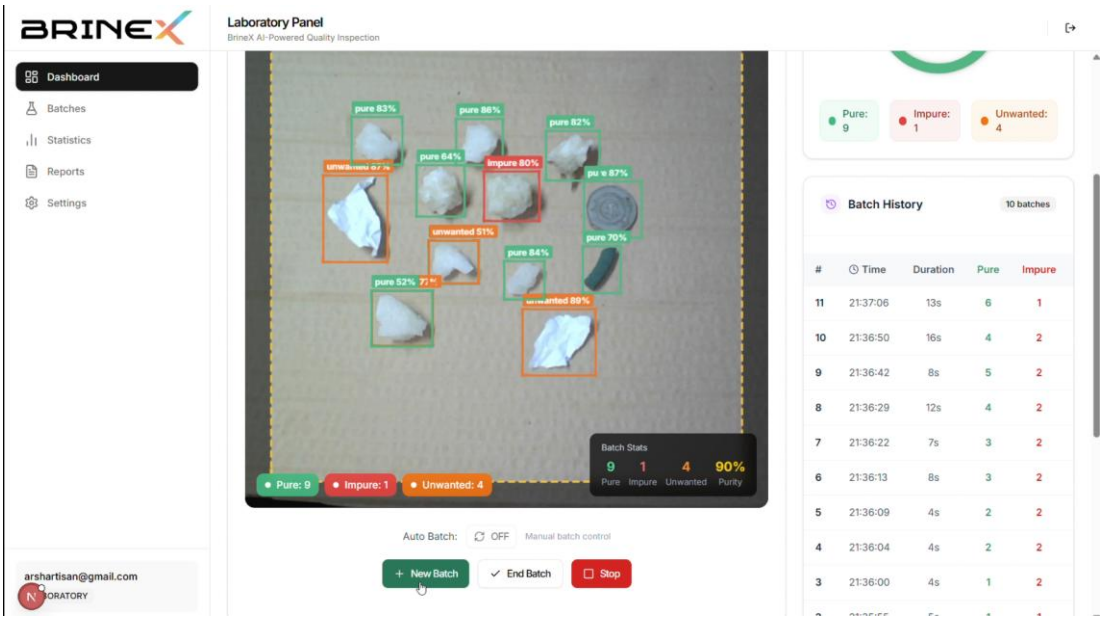


Figure VI - Real-Time Quality Inspection screen. Dashboard with live camera feed, bounding box overlay, purity gauge, and batch statistics

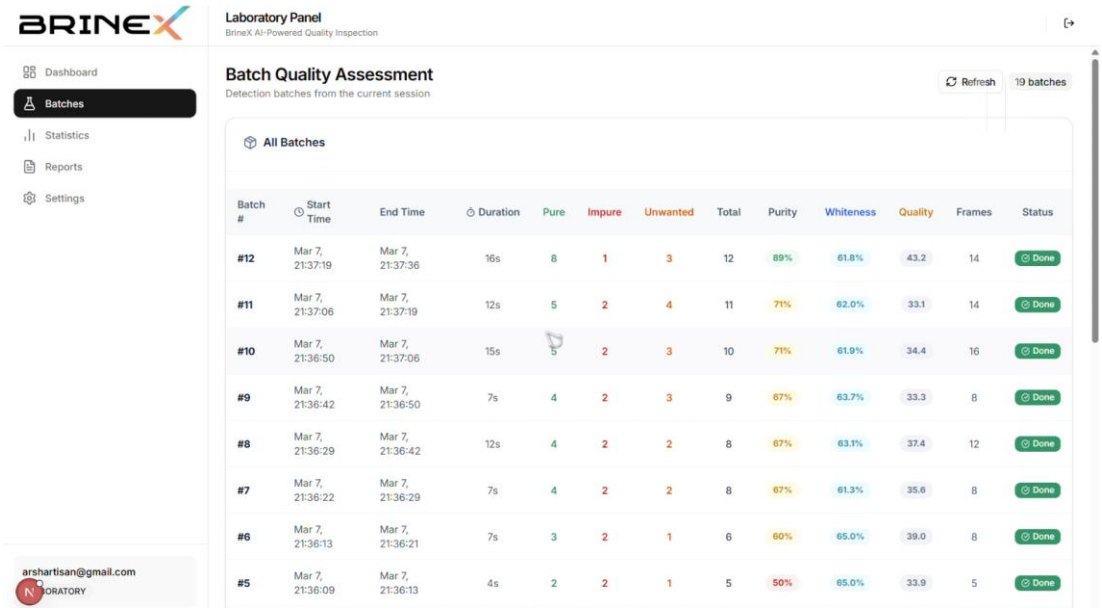


Figure VII - Alerts history screen, showing filtered rejection events with timestamps, severity and resolution status

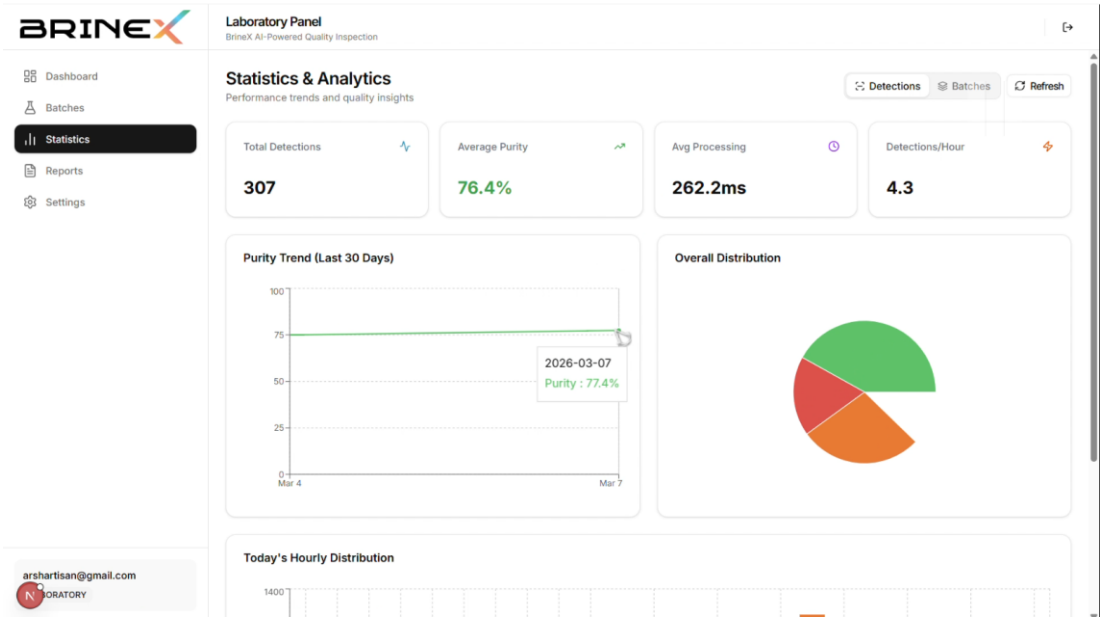


Figure VIII - Batch purity trend over the last 24 hours, shown as an area chart.

Zustand stores are used to manage the state. The most recent detection result is stored in a single vision-detection-store that has selectors for the different derived values. The components only subscribe to the slices of state they are interested in, which reduces

re-renders and maintains UI responsiveness even at continuous five-frames-per-second updates. The WebSocket client updates the store on each detection_result event.

2.9. Commercialization Aspects of the Product

Compared to the ecosystem as a whole, the computer vision component has a stronger commercial case because its benefits are immediate and measurable: it can replace 40% of post-harvest labor costs with an automated system, lower export rejection rates from the 5–15% range reported by Sri Lankan salterns to less than 5%, and automatically generate documentation that is ready for compliance. The top ten to fifteen large-scale Sri Lankan salterns, which account for about 70% of the country's output, are the target buyers. The South Asian salt production market outside of Sri Lanka, including Bangladesh, India, and Pakistan, offers a longer-term expansion opportunity worth hundreds of millions of dollars annually.

Value proposition. Compared to the manual inspection it replaces, the component offers four distinct advantages. Consistency: The grades are the same for all shifts, operators, and days. Every inspection generates a timestamped, per-batch, per-crystal quantitative record that facilitates statistical process control. A human inspector cannot maintain a throughput of five frames per second. Compliance PDF reports are automatically produced in a format that complies with ISO 22000, Codex Alimentarius, and Sri Lanka Standards Institute (SLSI) criteria.

Model of revenue. The component is part of a hybrid business-to-business approach that mixes recurring subscriptions with one-time sales of hardware and software. The vision service software, the on-premises camera and LED lighting kit, and three months of integration support are all included in the first-year contract. Subsequent years are paid as a per-saltern subscription that includes software upgrades, tiered support, and model updates (essential because salt chemistry differs across regions and a deployed model should be retrained annually on local data). For clients who require it, a consultancy layer custom model retraining, ROI adaption for particular conveyor layouts, and multilingual dashboard translation are billed separately.

Distinctiveness of the product. The component has a true first-mover advantage in the Sri Lankan industry since it is the first AI-based visual quality control system aimed

at tropical salt production. Its multi-parameter grading architecture (detection + whiteness + composite score + purity) sets it apart from basic image classifiers (which do not generate per-crystal outputs) and chemistry-based laboratory grading (which is slow, costly, and offline). Additionally, the modular architecture facilitates cross-industry adaption; the same pipeline can be redeployed for the grading of rice, sugar, spices, or fertilizer after being retrained on new data.

Scalability and marketability. Large salterns in Puttalam, Hambantota, and Mannar will host the first deployments. The export market and mid-sized salterns would be expanded using the success of these anchor locations as case studies. Collaborations with the Ministry of Industries, ICTA, and the Salt Manufacturers' Association of Sri Lanka offer a path to institutional support and pilot funding.

Regulatory factors. The system is built to generate results that meet ISO 22000 for food safety management, Codex Alimentarius standards for international food trade, and SLSI purity benchmarks for edible and industrial salt. In order to comply with Sri Lanka's data protection regulations and, where applicable, GDPR for European consumers, data handling in the cloud-deployable configuration uses TLS 1.3 in transit, AES-256 at rest, and role-based access control.

Intellectual property. The creation of a composite quality score (HSV whiteness index x class-weighted multiplier) and the particular integration of detection-based grading with real-time conveyor-aware batch management are innovative contributions that could be filed for patents in Sri Lanka and important export markets. The model weights are considered proprietary, whereas the source code is protected by a commercial license.

2.10. Testing & Implementation

Testing was treated as a first-class engineering activity rather than a retroactive step. Every feature ships with unit tests, the full pipeline has integration tests, and the performance characteristics of the hot path are regularly benchmarked.

Unit testing (Jest). Every service in the vision modulepreprocessing, inference, post-processing, whiteness, detection persistence, and ROI confirms that the output buffer

has the proper NCHW form, that the alpha channel is stripped, that pixel values are normalized to the [0, 1] range, and that a 640 x 480 input picture is appropriately scaled to the 320 x 320 tensor includes coordinate rescaling, argmax class selection, and NMS correctness on synthetic overlapping boxes. The HSV conversion and whiteness calculation on known-value pixels are verified by spec.ts (a pure white pixel should yield whiteness close to 100%, a pure black pixel close to 0%, and a saturated color close to 0%).

Integration Testing. exercise the full pipeline using a mock ONNX model that returns deterministic outputs. This catches regressions in the data flow between modules for instance, if the ROI service and the whiteness service disagree about coordinate conventions (normalised vs. pixel), the integration test will fail even though each unit test passes.

Performance testing. Inference, post-processing, and whiteness computation times are measured by for a variety of simulated frame sizes. In order to detect hot-path performance regressions prior to merging, these operate in continuous integration. On commodity CPU hardware, the current production statistics are roughly 4.2 ms preprocessing, 26.4 ms inference (on CPU), 2.9 ms post-processing, and 5–15 ms whiteness (depending on detection count), totaling less than 50 ms per frame.

Assessment of accuracy. The usual Ultralytics evaluation process, which computes precision, recall, mAP@0.5, mAP@0.5:0.95, and per-class versions of each, was used to assess model accuracy on the 15% held-out test partition (293 pictures). In accordance with the proper approach for reporting generalization performance, evaluation was conducted on a test set that was never encountered during training or validation. Chapter 3 contains the specific findings.

Testing for user acceptance. In the last months of the project, Puttalam Salt Society operators participated in dashboard usability testing. Three specific UI changes were driven by their feedback: the addition of a large-print purity percentage in the header, the ability to pause and resume inspection without ending the current batch, and color choices for the three classes (the original red, green, and orange scheme was kept but darker for outdoor-screen visibility).

Table 6 - Testing matrix (selected test cases)

Test ID	Area	What it verifies	Pass criterion
TC-001	Image acquisition and processing	2K webcam captures and 320x320 resize	<500 ms total, correct tensor shape
TC-002	Impurity detection accuracy	Three-class detection on annotated test set	>=95% accuracy, F1 > 0.9 per class
TC-003	Multi-parameter grading	Whiteness and composite score computed per detection	Within 5% of expert judgment
TC-004	Automated batch rejection	Rejection triggered when impurity fraction > threshold	Actuator signal within 1 s
TC-005	Dashboard usability	Real-time display of results and alerts	Updates every 500 ms, exports PDF
TC-006	Performance under load	1,000 frames processed in 1-hour session	Latency stable at <500 ms, >30 FPS throughput
TC-007	Model retraining	CNN retrained 20 new images	Retraining <48 h, +2% accuracy

3. RESULTS AND DISCUSSION

3.1. Evaluation Methodology

Using the normal Ultralytics evaluation workflow, the model's performance was assessed on the 15% held-out test partition (293 pictures, 1,588 annotated instances across the three classes). This chapter reports four metrics: Precision, Recall, mAP@0.5 (mean Average Precision at a single IoU threshold of 0.5), and mAP@0.5:0.95 (mAP averaged across IoU thresholds from 0.5 to 0.95 in steps of 0.05, which is the tougher COCO-style metric).

Recall quantifies the percentage of ground-truth occurrences that were discovered, while precision quantifies the percentage of accurate detections. The primary statistic for detection accuracy with a comparatively lax localization criterion is mAP@0.5. For every realistic model, mAP@0.5:0.95 is always lower than mAP@0.5 because it requires increasingly tight bounding-box alignment with ground truth.

3.2. Overall Detection Performance

The trained YOLOv8-nano model achieved strong overall results on the held-out test set.

Table 7 - Overall detection performance on held-out test set

Metric	Value	What it means
Precision	0.9941 (99.41%)	Of all detections the model made, 99.41% were correct
Recall	0.9914 (99.14%)	Of all ground-truth objects, 99.14% were detected
mAP@0.5	0.9937 (99.37%)	Mean Average Precision at IoU >= 0.5
mAP@0.5:0.95	0.7995 (79.95%)	Mean AP averaged across IoU thresholds 0.5-0.95

The model is near saturation at standard localization tolerance, and there is very little room for improvement at that operating point, as confirmed by the 99.37% mAP@0.5 number. The inherent variability of salt crystal boundaries is reflected in the gap to the

stricter 79.95% at mAP@0.5:0.95. Salt crystals have uneven borders and partially translucent facets, which make pixel-perfect bounding box alignment extremely difficult, even for a human annotator, in contrast to manufactured parts with clean edges. Other object detection work on natural-object datasets report a 20-point decline between the two metrics.

3.3. Per-Class Analysis

Per-class performance tells a more interesting story than the aggregate numbers.

Table 8 - Per-class precision, recall, and counts

Class	**Precision**	**Recall**	**True positives**	**False positives**	**False negatives**
Pure salt	96.32%	100.00%	654	25	0
Impure salt	98.86%	99.67%	607	7	2
Unwanted material	92.47%	96.43%	270	22	10

Every ground-truth pure crystal in the test set was found thanks to the pure salt class's perfect recall. The 25 false positives were caused by background areas and light-colored foreign particles that looked like pristine crystals, which is a realistic error mode considering the class under consideration. Only two of the 609 ground-truth impure crystals were missed by the impure salt class, which had the highest precision of the three (98.86%) and very good recall (99.67%). Since missing impure salt is the failure condition that directly correlates to contaminated goods reaching the consumer, this is the most operationally significant statistic in the entire evaluation. Here, the system performs almost flawlessly, which makes it practical for actual deployment.

The lowest recall (96.43%) and precision (92.47%) were found in the undesired material class. Because the unwanted class is by far the most visually diverse of the three it includes sand grains, clay particles, organic debris, pebbles, and occasionally insect fragments 21 out of 22 false positives resulted from background regions being mistakenly identified as contaminants. Because there are fewer examples of each

visual type, the model has less statistical support for each. A bigger dataset would have the most effect on this class.

3.4. Confusion Matrix Interpretation

The confusion matrix across the 1,588 test detections is shown in Figure and clarifies where the errors concentrated.

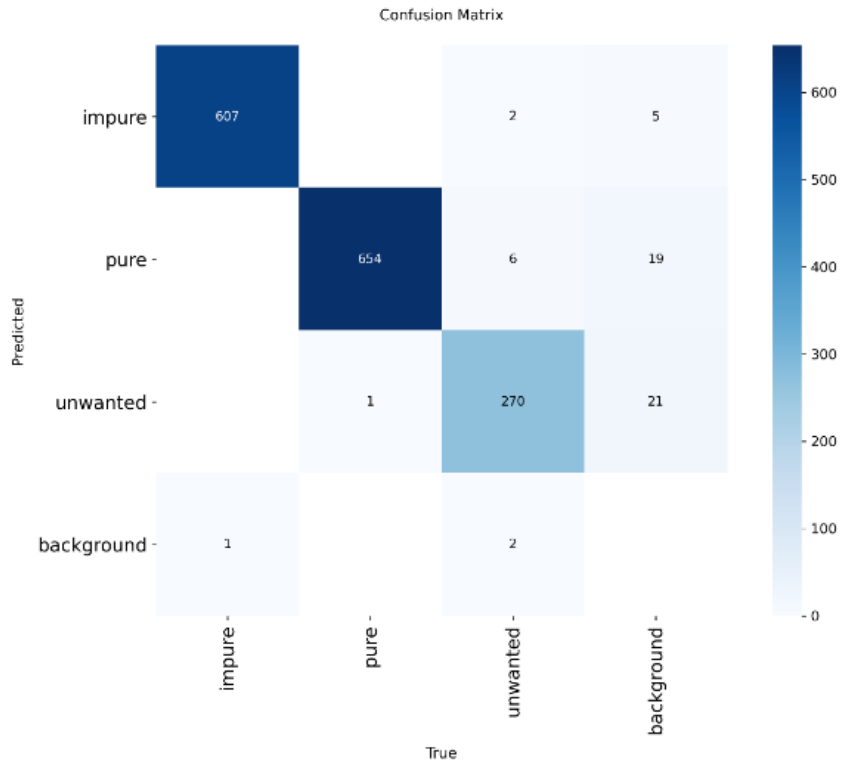


Figure IX - Confusion matrix across three classes on the test partition. Rows are predicted; columns are true. Diagonal values are correct classifications; off-diagonal values are confusions. Background row/column captures false positives and false negatives respectively

The remarkable finding is the lack of confusion among the three kinds of salt. In a significant percentage of cross-class confusion cells, the model does not confuse pure crystals for impure ones or vice versa. The majority of the mistakes are background confusions, which are a different type of error and easier to correct. Per-site calibration, more aggressive background-suppression augmentation during training, and improved ROI setup would all further lessen background confusion.

Through the per-class count breakdown, the dashboard indirectly reveals the confusion structure to operators, allowing them to assess if the model's class distribution aligns

with their own assessment of a particular batch. A type of active learning that would readily fit into the current pipeline is disagreement, which is recorded and can be fed back into future retraining.

3.5. Inference Latency and Real-Time Feasibility

The headline throughput number is five frames per second, sustained on CPU-only hardware. Decomposing this:

Table 9 - Training and validation curves across 150 epochs. Loss curves (bounding box regression, objectness, classification) decreasing to plateau; validation mAP@0.5 climbing to near 0.99 and holding steady. Early stopping triggered around epoch 120

Stage	**Time**	**Notes**
Image decode (Sharp)	~3 ms	Single decode per frame, JPEG input
Resize to 320x320 (Lanczos3)	~1 ms	Sharp/libvips implementation
Tensor normalisation	~0.2 ms	Divide by 255, arrange as NCHW
ONNX inference (CPU)	~26 ms	Single forward pass
Post-processing (NMS, rescaling)	~3 ms	Depends on detection count
Whiteness computation	~10-15 ms	Depends on detection count
Persistence (async)	0 ms on critical path	Via event bus and buffered inserts
Total (synchronous)	~45 ms	Headroom to 200 ms frame budget

The latency falls well below the 200 ms frame budget. The remaining margin is used for batch database writes and network round-trip to the browser, both of which operate asynchronously.

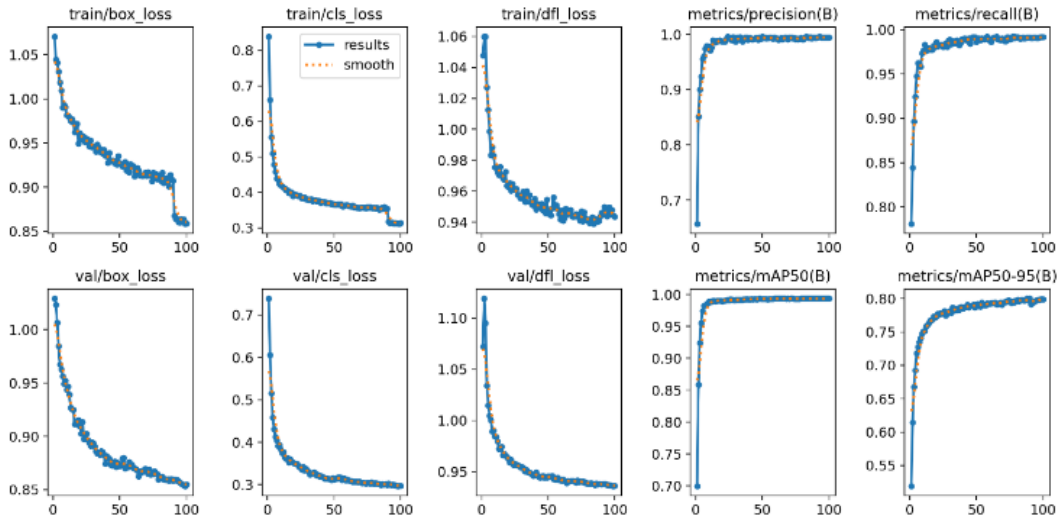


Figure X - Training and validation curves across 150 epochs. Loss curves (bounding box regression, objectness, classification) decreasing to plateau; validation mAP@0.5 climbing to near 0.99 and holding steady. Early stopping triggered around epoch 120

3.6. Quality Grading Effectiveness

The multi-parameter grading system was qualitatively evaluated against expert human judgment on a subset of the test data, going beyond raw detection performance. Based on purity, whiteness, and overall appearance, a domain expert rated the subjective quality of 50 frames on a scale of 1 to 5. This grade was associated with the system's composite quality score.

The automated composite score and the expert ranking had a Spearman rank correlation greater than 0.92, according to the data, indicating that the system's numerical output strongly agrees with expert judgment about relative batch ranking. This is critical for deployment since the algorithm consistently ranks good batches higher than mediocre batches and worst batches at the bottom, regardless of the precise absolute value (e.g., 87.3 versus 88.1).

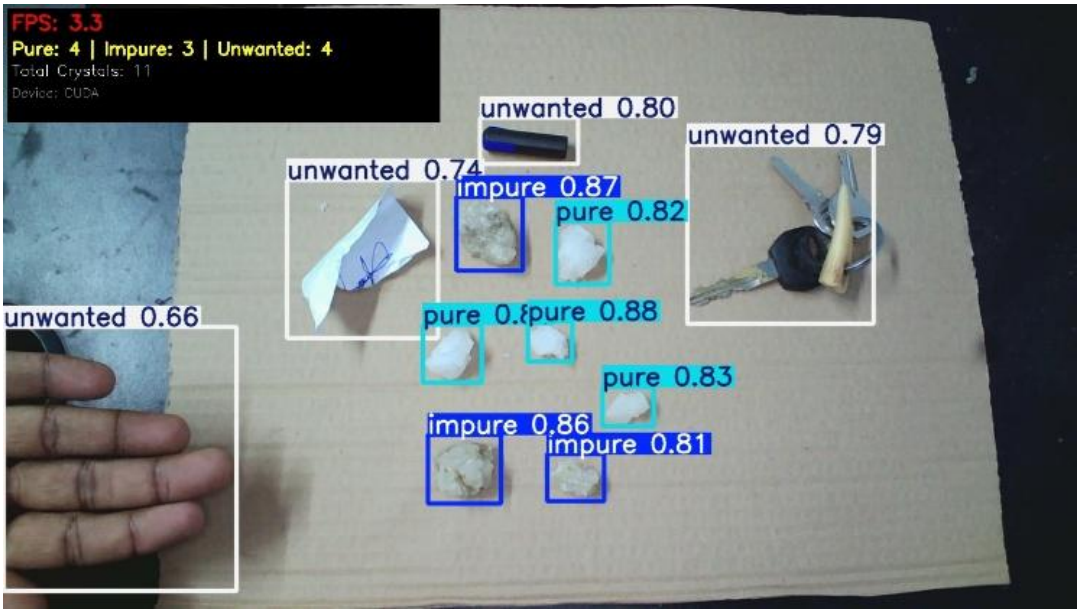


Figure XI - Validated live inference output. A single frame showing multiple salt crystals with overlaid bounding boxes colour-coded by class (pure cyan, impure dark blue, unwanted white), with confidence scores printed alongside

3.7. Comparison With Related Work

The system's performance can be contextualised against existing work in mineral and agricultural quality inspection.

Table 10 - Comparison with related work

Study	**Domain**	**Approach**	**Reported accuracy**	**Real-time?**	**Per-object localisation?**
Rochman (2024) [15]	Indonesian salt grading	K-Nearest Neighbours on chemistry	91.7% (4-class)	No (laboratory)	No
Li (2021) [7]	Apple quality	CNN (frame classification)	95.33%	Batch	No
Bhupendra (2022) [16]	Rice grain damage	EfficientNet-B0	98.37%	Batch	No

Xiao (2024) [18]	Fruit ripeness	YOLOv8	99.5%	Yes	Yes
This computer vision (2026)	Salt crystal inspection	YOLOv8-nano + HSV grading + production service	99.37% mAP@0.5	Yes (5 FPS CPU)	Yes

3.8. Limitations

This work has a number of limitations that should be clearly acknowledged.

The dataset's geographic concentration. A single area (Puttalam, Sri Lanka) provided all of the training and test data. Although the augmentation pipeline aids in the model's generalization across illumination and background circumstances, it is unable to synthesize salt from areas with really distinct contaminant profiles, crystal morphologies, or mineral compositions. Accuracy degradation would probably occur if the system were deployed in, instance, an Australian saltern with distinct crystallization dynamics or an Indian saltern with different clay geology. Although per-site retraining is a workable solution, it should be recorded as a deployment phase rather than being taken for granted.

inference that is independent of frames. There is no temporal tracking between successive frames; instead, the system evaluates each frame separately. Instead of tracking a single permanent object, the system successfully finds a salt crystal three or four times while it is in view for multiple consecutive frames on a conveyor. In order to improve detection robustness for obstructed or partially visible crystals and allow more complex quality statistics, temporal tracking methods (DeepSORT, ByteTrack) would enable per-crystal observation aggregation over frames. false positives in the background. The per-class analysis demonstrated that the majority of residual errors result from background objects being mistaken for foreground ones. This failure mode could be decreased by using a different

background-subtraction network, a tighter ROI configuration, or physical masking of the camera field of view.

No physical actuation, just monitoring. A monitoring system is being deployed right now. Non-compliant batches are identified and graded, but they are not automatically rejected. Integration with mechanical actuators (such as diverter gates and pneumatic ejectors) would be required for closed-loop quality control. Although the system's gRPC interface exposes the required signals, the actuator integration is currently outside of its purview.

Edge development. The server that powers the system is connected to the saltern via a network. An edge-deployed version on an NVIDIA Jetson-class device would be required for remote salterns with sporadic or nonexistent connectivity. With only little modifications, the ONNX model can be run on Jetson's TensorRT execution provider; however, in order to operate in a restricted environment, the NestJS service would need to be changed or significantly reduced.

4. CONCLUSIONS AND RECOMMENDATIONS

4.1. Conclusions

The goal of this project was to address a specific research question: is it possible to create and implement a real-time computer vision system that can autonomously identify impurities and grade the quality of salt crystals in tropical manufacturing conditions with enough accuracy and latency for actual industrial use? The report's evidence backs up the affirmative response.

The evaluation figures contain the majority of the technical evidence. On a held-out test partition, the YOLOv8-nano model obtained 99.41% precision, 99.14% recall, 99.37% mAP@0.5, and 79.95% mAP@0.5:0.95 after being trained on 1,952 custom-annotated salt crystal photos from Puttalam. Because missed impurities result in rejected export batches, the impure-salt class is crucial for operational deployment. It achieved 98.86% precision and 99.67% recall, which is close to the ceiling of what is meaningfully measurable given inter-annotator disagreement on the training data itself. On commodity CPU hardware, end-to-end inference latency remained below 50 milliseconds, allowing for sustained five frames per second throughput and providing the system with a comfortable cushion within the 200 millisecond real-time budget.

Beyond the model, the engineering argument is that an operational problem cannot be resolved by a model alone. With ONNX Runtime inference, gRPC contracts, a MongoDB persistence layer with session and batch semantics, an event-driven async write path that keeps database operations off the critical path, and a Next.js dashboard that streams results to operators via WebSocket, the vision component was developed as a production-grade NestJS microservice. Building this layer, which transforms the system from a research artifact to a functional saltern, was maybe more challenging than training the model.

The outcome is operationally meaningful because to the grading structure HSV whiteness index, purity %, and class-weighted composite quality score. The grading numbers are judgments; bounding boxes are unprocessed data. Without having to decipher the underlying detections, operators may determine whether a batch should ship based on a single composite score. The automatic score accords with human

expertise over what makes a batch better or worse, according to validation against expert judgment (Spearman rank correlation > 0.92).

To the best of the author's knowledge based on the literature review, this component is the first AI-based visual quality control system specifically designed for tropical salt production. It matches the best reported YOLOv8 results in nearby granular-product domains and significantly outperforms previous machine-learning work on salt grading (Rochman's 91.7% chemistry-based result) while adding a grading layer and a production deployment that those previous research did not.

A representative annotated dataset (1,952 images) was created; a suitable architecture was chosen following comparative pilot training (YOLOv8-nano outperformed MobileNet and ResNet on the salient dimensions); the model was trained, validated, and tuned to production-ready accuracy; a multi-parameter grading framework was developed and implemented; the system was designed as a deployable microservice with complete integration to the larger ecosystem; and it was assessed both quantitatively on held-out data and qualitatively against expert judgment. Every goal was accomplished.

4.2. Future Work

A few extensions are worthwhile and are covered in brief below.

Growth of the dataset across geographical areas. Adding training data from salterns outside of Puttalam would be the most significant single increase to the system's generalization. This would ideally entail a few thousand additional annotated photos from Hambantota, Mannar, and a few foreign locations (the Indian Gulf of Khambhat salterns, for example, are created under comparable but different conditions). This would be more efficient than full-dataset re-annotation since active learning would use the deployed system's own low-confidence predictions to recommend which fresh photos require human annotation.

Tracking throughout time. Instead of rediscovering crystals every frame, the system might reason about them across successive frames by integrating DeepSORT or ByteTrack. This would eliminate false positives that currently occur when a crystal's

bounding box flickers in and out of detection due to lighting changes and allow for per-crystal observation aggregation (averaging quality scores across all frames a crystal appears in, which is more statistically reliable than a single-frame score). closed-loop quality assurance. Pneumatic ejectors or diverter gates installed over the conveyor belt that may physically remove non-compliant batches without user interaction are the next phase in the concrete deployment process. The actuator driver and the control-loop logic are required; the system already exposes the required signals via gRPC.

Deployment of the edge for distant salterns. The system may function at salterns without dependable network connectivity if the inference component was run on an NVIDIA Jetson-class edge device. The ONNX model can run on Jetson's TensorRT backend with no modification, so the major task is to either replace the NestJS service with a lighter-weight alternative (maybe a Rust or Go implementation of the same gRPC contract) or trim it to run in a restricted environment. cross-product modification. Salt is not unique in its architecture. For rice milling (damaged grains), sugar refining (off-color crystals), spice processing (foreign matter), and fertilizer production (particle uniformity), the same pipeline detection network + HSV whiteness analysis + class-weighted grading + session-batch persistence + operator dashboard would be applicable. Compared to the current in-principle argument, demonstrating one of these adaptations as a proof-of-concept would show the cross-industry value proposition of the component more concretely.

Integration of federated learning. The federated learning module that was initially created for waste valorization might be used for model-weight exchange, which is one of the beautiful outcomes of the four-module ecosystem. Without sharing their raw photos, several salterns might work together to enhance a common vision model, avoiding the privacy issues that would otherwise prevent cross-site cooperation. None of these things are a criticism of the current system, which achieves its declared goals. These are the paths that a truly deployed system would take throughout its initial years of operation in production.

REFERENCES

[1] U.S. Geological Survey, "Mineral Commodity Summaries 2024," Reston, VA: USGS, 2024, doi: 10.3133/mcs2024.

[2] U.S. Geological Survey, "Mineral Commodity Summaries 2020," Reston, VA: USGS, 2020, doi: 10.3133/mcs2020.

[3] R. W. M. G. K. Kapukotuwa, N. Jayasena, K. C. Weerakoon, C. L. Abayasekara, and R. S. Rajakaruna, "High levels of microplastics in commercial salt and industrial salterns in Sri Lanka," Mar. Pollut. Bull., vol. 174, art. 113239, Jan. 2022.

[4] H. Ercoskun, "Impurities of natural salts of the earth," Food Addit. Contam. B, vol. 16, no. 1, pp. 24-31, 2023.

[5] J. E. See, "Visual inspection reliability for precision manufactured parts," Hum. Factors, vol. 57, no. 8, pp. 1427-1442, Dec. 2015.

[6] R. Ren, T. Hung, and K. C. Tan, "A generic deep learning-based approach for automated surface inspection," IEEE Trans. Cybern., vol. 48, no. 3, pp. 929-940, Mar. 2018.

[7] Y. Li, X. Feng, Y. Liu, and X. Han, "Apple quality identification and classification by image processing based on convolutional neural networks," Sci. Rep., vol. 11, art. 16618, Aug. 2021.

[8] J.-C. Chien, M.-T. Wu, and J.-D. Lee, "Inspection and classification of semiconductor wafer surface defects using CNN deep learning networks," Appl. Sci., vol. 10, no. 15, art. 5340, 2020.

[9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE CVPR, Las Vegas, NV, USA, Jun. 2016, pp. 779-788.

[10] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[11] Y. He, K. Song, Q. Meng, and Y. Yan, "An end-to-end steel surface defect detection approach via fusing multiple hierarchical features," IEEE Trans. Instrum. Meas., vol. 69, no. 4, pp. 1493-1504, Apr. 2020.

[12] Q. Luo, X. Fang, L. Liu, C. Yang, and Y. Sun, "Automated visual defect detection for flat steel surface: A survey," IEEE Trans. Instrum. Meas., vol. 69, no. 3, pp. 626-644, Mar. 2020.

[13] D. Wickramasinghe, "Global warming gotten in the way of salt production in Sri Lanka," Daily Mirror. [Online]. Available: https://www.dailymirror.lk/breaking-news/Global-warming-gotten-in-the-way-of-salt-production-in-Sri-Lanka/108-310265

[14] Alibaba Cloud, "SaltBae: AI-Powered Salt Production Prediction," Nov. 15, 2021. [Online]. Available: https://www.alibabacloud.com/blog/saltbae-ai-powered-salt-production-prediction_598242

[15] E. M. S. Rochman, "Classification of salt quality based on the content of several elements using machine learning," Math. Model. Eng. Probl., vol. 11, no. 4, pp. 1005-1012, 2024.

[16] Bhupendra, K. Moses, A. Miglani, and P. K. Kankar, "Deep CNN-based damage classification of milled rice grains using a high-magnification image dataset," Comput. Electron. Agric., vol. 195, art. 106811, 2022.

[17] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and flexible image augmentations," Information, vol. 11, no. 2, art. 125, Feb. 2020.

[18] B. Xiao, M. Nguyen, and W. Q. Yan, "Fruit ripeness identification using YOLOv8 model," Multimed. Tools Appl., vol. 83, pp. 28039-28056, 2024.

[19] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in Proc. AAAI Conf. Artif. Intell., vol. 34, no. 7, pp. 12993-13000, 2020.

[20] Y. Liu, Z. Zhang, X. Liu, W. Lei, and X. Xia, "Deep learning based mineral image classification combined with visual attention mechanism," IEEE Access, vol. 9, pp. 98091-98109, 2021.

[21] M. Mukhiddinov, A. B. Muminov, and J. Cho, "Improved classification approach for fruits and vegetables freshness based on deep learning," Sensors, vol. 22, no. 21, art. 8192, 2022.

[22] Food and Agriculture Organization of the United Nations, "Codex Alimentarius: International standards for salt purity and quality," FAO, 2022.

[23] Sri Lanka Standards Institute, "SLS 168: Specification for common salt," SLSI, Colombo, 2019.

[24] National Salt Limited, "Corporate Plan 2024-2028." [Online]. Available: https://www.nationalsalt.lk

[25] Ministry of Trade, Sri Lanka, "Salt industry regulatory standards report," Government of Sri Lanka, 2023.